SERVICIO WEB

¿Qué haremos?

- Proyecto: SOAP con Spring Boot (starter-web + starter-web-services)
- Packaging: WAR ejecutable (java -jar target/*.war)
- Creación de XML de respuesta usando org.w3c.dom (DOMImplementationRegistry) — sin javax ni jakarta.
- Probar con SoapUI (pasos detallados abajo).
- WSDL público en: http://localhost:8080/ws/countries.wsdl
- Endpoint (POST): http://localhost:8080/ws

Paso 0 — Requisitos (antes de empezar)

- JDK 17 instalado. Verifica:

```
java -version
```

- STS4 (opcional) o cualquier IDE que uses.
- SoapUI Desktop instalado (para pruebas).

Paso 1 — Crear el proyecto en STS4 (wizard)

1. File → New → Spring Starter Project.
2. Rellena:
    - Group: com.example
    - Artifact: soapws
    - Name: soapws
    - Packaging: WAR
    - Java: 17
3. En Dependencies selecciona:
    - Spring Web
    - Spring Web Services
4. Finish.

Paso 2 — Reemplazar pom.xml (ruta:

pom.xml en la raíz del proyecto)

Pega exactamente esto:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                  https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>soapws</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.4</version>
    <relativePath/>
  </parent>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <!-- web + web services -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
```

```
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web-services</artifactId>
  </dependency>

  <!-- WSDL helper -->
  <dependency>
    <groupId>wsdl4j</groupId>
    <artifactId>wsdl4j</artifactId>
    <version>1.6.3</version>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
    <plugin>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>17</source>
        <target>17</target>
      </configuration>
```

```
    </plugin>
  </plugins>
 </build>
</project>
```

Paso 3 — Crear la estructura y archivos (rutas exactas)

Crea estos archivos exactamente en las rutas indicadas (respetando paquete com.example.soapws).

3.1 src/main/java/com/example/soapws/SoapWsApplication.java

```
package com.example.soapws;


import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.web.servlet.support.SpringBootServletInitializer;


@SpringBootApplication
public class SoapWsApplication extends SpringBootServletInitializer {


  @Override
  protected SpringApplicationBuilder configure(SpringApplicationBuilder builder) {
    return builder.sources(SoapWsApplication.class);
  }


  public static void main(String[] args) {
    SpringApplication.run(SoapWsApplication.class, args);
  }
```

```
}
```

3.2 src/main/java/com/example/soapws/WebServiceConfig.java

```
package com.example.soapws;

import org.springframework.boot.web.servlet.ServletRegistrationBean;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.io.ClassPathResource;
import org.springframework.ws.transport.http.MessageDispatcherServlet;
import org.springframework.xml.xsd.SimpleXsdSchema;
import org.springframework.xml.xsd.XsdSchema;
import org.springframework.ws.wsdl.wsdl11.DefaultWsdl11Definition;

@Configuration
public class WebServiceConfig {

  @Bean
  public ServletRegistrationBean<MessageDispatcherServlet>
messageDispatcherServlet(ApplicationContext ctx) {
    MessageDispatcherServlet servlet = new MessageDispatcherServlet();
    servlet.setApplicationContext(ctx);
    servlet.setTransformWsdlLocations(true);
    return new ServletRegistrationBean<>(servlet, "/ws/*");
  }

  // El bean se llama "countries", por eso el WSDL estará en /ws/countries.wsdl
```

```
    @Bean(name = "countries")
    public DefaultWsdl11Definition defaultWsdl11Definition(XsdSchema
countriesSchema) {
        DefaultWsdl11Definition def = new DefaultWsdl11Definition();
        def.setPortTypeName("CountriesPort");
        def.setLocationUri("/ws");
        def.setTargetNamespace("http://example.com/soapws");
        def.setSchema(countriesSchema);
        return def;
    }

    @Bean
    public XsdSchema countriesSchema() {
        return new SimpleXsdSchema(new ClassPathResource("countries.xsd"));
    }
}
```

3.3 src/main/java/com/example/soapws/Country.java

```
package com.example.soapws;

public class Country {
    private String name;
    private String capital;
    private int population;
    private String currency;

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
```

```
    public String getCapital() { return capital; }
    public void setCapital(String capital) { this.capital = capital; }
    public int getPopulation() { return population; }
    public void setPopulation(int population) { this.population = population; }
    public String getCurrency() { return currency; }
    public void setCurrency(String currency) { this.currency = currency; }
}
```

3.4 src/main/java/com/example/soapws/CountryRepository.java

```
package com.example.soapws;

import org.springframework.stereotype.Component;
import java.util.HashMap;
import java.util.Map;

@Component
public class CountryRepository {

    private static final Map<String, Country> countries = new HashMap<>();

    public CountryRepository() {
        Country spain = new Country();
        spain.setName("Spain");
        spain.setCapital("Madrid");
        spain.setPopulation(46704314);
        spain.setCurrency("EUR");
        countries.put("Spain", spain);

        Country peru = new Country();
```

```
        peru.setName("Peru");
        peru.setCapital("Lima");
        peru.setPopulation(34000000);
        peru.setCurrency("PEN");
        countries.put("Peru", peru);
    }

    public Country findCountry(String name) {
        return countries.get(name);
    }
}
```

3.5 EndPoint: src/main/java/com/example/soapws/CountryEndpoint.java

```
package com.example.soapws;

import org.springframework.ws.server.endpoint.annotation.Endpoint;
import org.springframework.ws.server.endpoint.annotation.PayloadRoot;
import org.springframework.ws.server.endpoint.annotation.RequestPayload;
import org.springframework.ws.server.endpoint.annotation.ResponsePayload;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.bootstrap.DOMImplementationRegistry;
import org.w3c.dom.DOMImplementation;

@Endpoint
```

```
public class CountryEndpoint {


    private static final String NAMESPACE = "http://example.com/soapws";

    private final CountryRepository repository;


    public CountryEndpoint(CountryRepository repository) {

        this.repository = repository;

    }


    @PayloadRoot(namespace = NAMESPACE, localPart =
"getCountryRequest")

    @ResponsePayload

    public Element getCountry(@RequestPayload Element request) throws
Exception {

        // Leer <name> (soporta con o sin namespace)

        String name = null;

        if (request.getElementsByTagNameNS(NAMESPACE,
"name").getLength() > 0) {

            name = request.getElementsByTagNameNS(NAMESPACE,
"name").item(0).getTextContent();

        } else if (request.getElementsByTagName("name").getLength() > 0) {

            name =
request.getElementsByTagName("name").item(0).getTextContent();

        }


        Country country = repository.findCountry(name);
```

```
// Crear documento XML usando las APIs del JDK
(DOMImplementation)

DOMImplementationRegistry registry =
DOMImplementationRegistry.newInstance();

// Obtener implementación para XML

DOMImplementation impl = registry.getDOMImplementation("XML
1.0");

Document doc = impl.createDocument(NAMESPACE,
"getCountryResponse", null);



Element responseEl = doc.getDocumentElement(); // ya es
<getCountryResponse>

Element countryEl = doc.createElementNS(NAMESPACE, "country");



Element nameEl = doc.createElementNS(NAMESPACE, "name");

nameEl.setTextContent(country != null ? country.getName() : "");

Element capitalEl = doc.createElementNS(NAMESPACE, "capital");

capitalEl.setTextContent(country != null ? country.getCapital() : "");

Element populationEl = doc.createElementNS(NAMESPACE,
"population");

populationEl.setTextContent(country != null ?
String.valueOf(country.getPopulation()) : "0");

Element currencyEl = doc.createElementNS(NAMESPACE,
"currency");

currencyEl.setTextContent(country != null ? country.getCurrency() : "");
```

```
        countryEl.appendChild(nameEl);
        countryEl.appendChild(capitalEl);
        countryEl.appendChild(populationEl);
        countryEl.appendChild(currencyEl);

        responseEl.appendChild(countryEl);
        return responseEl;
    }
}
```

Paso 4 — XSD (archivo que define el contrato)

Ruta: src/main/resources/countries.xsd — crea exactamente este archivo:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           targetNamespace="http://example.com/soapws"
           xmlns:tns="http://example.com/soapws"
           elementFormDefault="qualified">

  <xs:element name="getCountryRequest">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
```

```
  </xs:element>


  <xs:element name="getCountryResponse">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="country" minOccurs="0">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="name" type="xs:string"/>
              <xs:element name="capital" type="xs:string"/>
              <xs:element name="population" type="xs:int"/>
              <xs:element name="currency" type="xs:string"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>


</xs:schema>
```

Paso 5 — Compilar y empaquetar (línea de comandos o STS)

Desde la raíz del proyecto:

mvn clean package

Paso 6 — Ejecutar el WAR (servidor embebido)

Ejecuta:

```
java -jar target/soapws-0.0.1-SNAPSHOT.war
```

En la consola verás que Spring Boot arranca. URL base por defecto: http://localhost:8080

- WSDL: http://localhost:8080/ws/countries.wsdl

  (la URL contiene el nombre del bean countries definido en WebServiceConfig).

Paso 7 — Probar con SoapUI (paso a paso, para alumnos)

1. Abrir SoapUI (versión desktop).
2. File → New SOAP Project.
   - Project Name: Countries
   - Initial WSDL: http://localhost:8080/ws/countries.wsdl
   - Marcar Create Requests (opcional) y click OK.
3. SoapUI mostrará el servicio y la operación getCountryRequest.
4. Abrir la request (dentro del proyecto). Verás una plantilla. Reemplázala por:

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
            xmlns:gs="http://example.com/soapws">
 <soapenv:Header/>
 <soapenv:Body>
  <gs:getCountryRequest>
   <gs:name>Spain</gs:name>
  </gs:getCountryRequest>
 </soapenv:Body>
</soapenv:Envelope>
```

5. Click Submit (▶).
6. Observa la pestaña Response. Debe devolver:

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
 <soapenv:Body>
  <ns2:getCountryResponse xmlns:ns2="http://example.com/soapws">
   <ns2:country>
```

```
      <ns2:name>Spain</ns2:name>
      <ns2:capital>Madrid</ns2:capital>
      <ns2:population>46704314</ns2:population>
      <ns2:currency>EUR</ns2:currency>
    </ns2:country>
  </ns2:getCountryResponse>
 </soapenv:Body>
</soapenv:Envelope>
```


7. En Raw puedes ver el Request/Response HTTP (headers, body). El Content-Type será text/xml;charset=UTF-8.

<u>Qué hace cada archivo:</u>

- pom.xml → dependencias y empaquetado.
- SoapWsApplication → arranca la aplicación.
- WebServiceConfig → registra el servlet SOAP y expone WSDL.
- countries.xsd → define el contrato (request/response).
- CountryEndpoint → lee la petición y construye la respuesta (a mano, con DOM).
- CountryRepository → datos “simulados” en memoria.